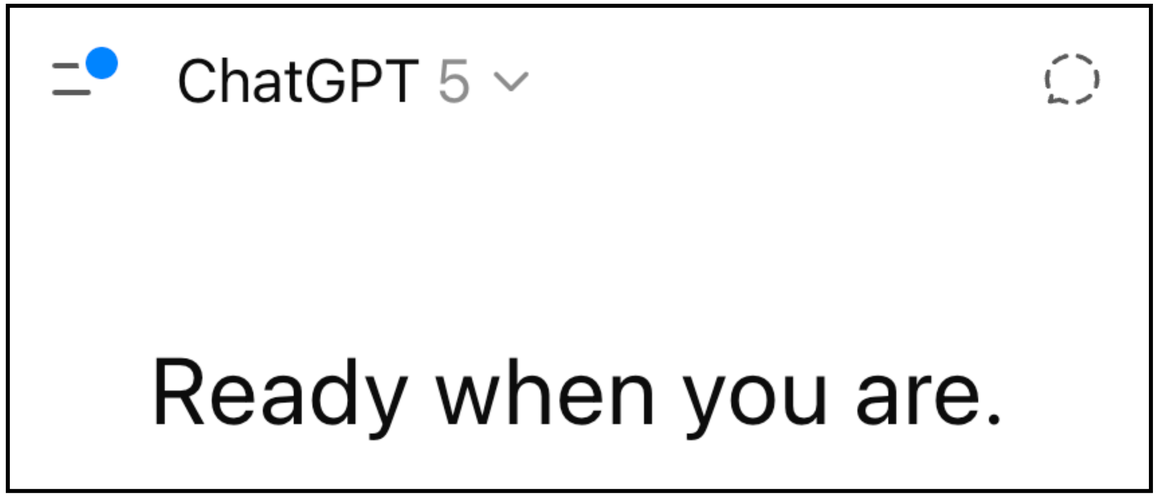
Lecture 11: Foundation Model Pre-training

LLMs, Pre-training, and Scaling Laws I

Speaker: Omar Khattab



Adapts content from Tatsu Hashimoto, Percy Liang, John Hewitt, Chris Manning, and others credited.

Outline

1. Foundation Models
2. Architecture for LLMs
3. Pre-training: Data, Hardware, and Runs
4. Scaling Laws I (motivation — to be continued)

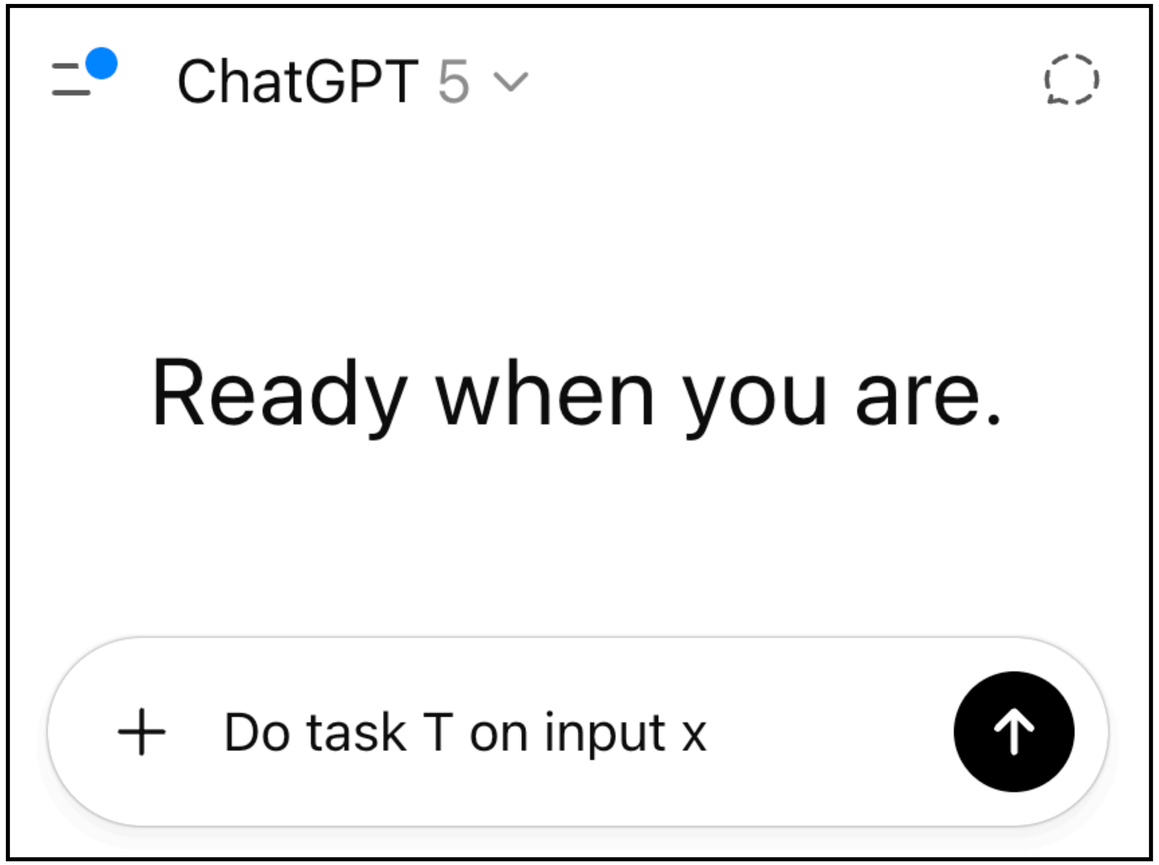
The goal of this lecture is to build some intuition (breadth) on pre-training.

What are foundation models?

Not long ago, if you wanted a model to do task T, you would…

1. collect lots of labeled data for T (e.g., supervised learning)
2. design a suitable architecture
3. initialize it randomly
4. train and validate

But now…



This shift happened largely over the past 3–8 years.

What might we want from foundation models?

1. **Language:** Understand our requests**.**
2. **Knowledge:** Know enough about the world to be useful.
3. **Retrieval and Tool Use:** Search for information if it’s not sure.
4. **In-Context Learning:** Learn new skills quickly from examples.
5. **Alignment:** Actually follow our instructions.
6. **Capability:** Be effective at following them.
7. **Safety:** But don’t do destructive things, even if asked.
8. **Reasoning:** Think longer, consider alternatives, and correct itself if needed.
9. …

How?

Idea #1: Reinforcement learning

- Put the model out into the world. Maybe as a robot?
- Give it rewards for the right behavior, e.g. following instructions.

Challenge?

- For a very very long time, it's just doing random things…
- It has no idea what different instructions mean.
- Even when it gets some occasional rewards, can it interpret and extrapolate them correctly?

Language provides a natural domain for the study of artificial intelligence, as the vast majority of reasoning tasks can be efficiently expressed and evaluated in language, and the world's text provides a wealth of data for unsupervised learning via generative modeling. Deep learning has recently seen rapid progress in language modeling, with state of the art models [RNSS18, DCLT18,

Scaling Laws for Neural Language Models (2020)

Idea #2: Supervised learning

- If just want a "*language* model", collect supervised question–answer pairs?
- Where do find good questions? How do we answer them? How many??

Dataset	Questions	Example
SQuAD ('16)	**100,000** human-crafted questions, about 500 Wikipedia pages	*Which NFL team represented the AFC at Super Bowl 50?*
HotPotQA ('18)	**100,000** human-crafted questions, about *pairs* of Wikipedia pages	*At which university does the biographer of John Clare teach?*
MS MARCO ('16)	**1,000,000** Bing queries	*what is the agenda for hollande's state visit to washington*
Natural Questions ('19)	**100,000** Google queries	can you make and receive calls in airplane mode

Challenges?

Idea #3: Self-supervised learning

What if we could scale the tokens we learn from by millions or billions of times?

Language Modeling (*next token prediction*) on… "the whole Web"?

Large Language Models in Machine Translation *(2007)*

Thorsten Brants **Ashok C. Popat** **Peng Xu** **Franz J. Och** **Jeffrey Dean**

Google, Inc.
1600 Amphitheatre Parkway
Mountain View, CA 94303, USA
{brants,popat,xp,och,jeff}@google.com

Abstract

This paper reports on the benefits of large-scale statistical language modeling in machine translation. A distributed infrastructure is proposed which we use to train on up to 2 trillion tokens, resulting in language models having up to 300 billion n-grams. It is capable of providing smoothed probabilities for fast, single-pass decoding. We introduce a new smoothing method, dubbed *Stupid Backoff*, that is inexpensive to train

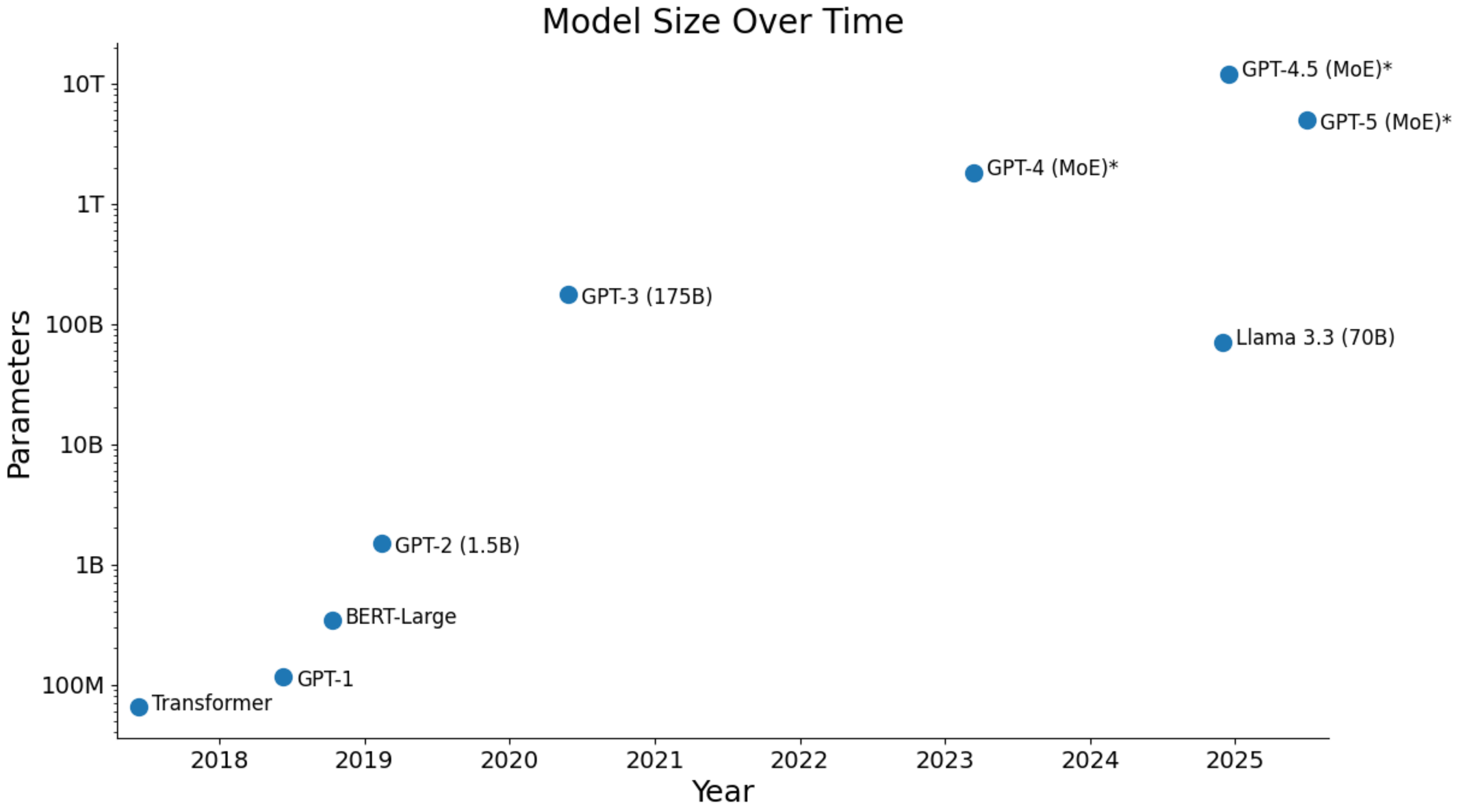
How might one build a language model that allows scaling to very large amounts of training data? (2) How much does translation performance improve as the size of the language model increases? (3) Is there a point of diminishing returns in performance as a function of language model size?

This paper proposes one possible answer to the first question, explores the second by providing learning curves in the context of a particular statistical machine translation system, and hints that the third may yet be some time in answering. In particular, it proposes a *distributed* language model training

People have trained "LLMs" at scale for 20 years…

What makes a modern LLM like GPT-5 so powerful?

(Many things! But Ingredient #1: Deep Learning.)



Very approximate (and speculative*) plot generated, with lots of iteration, by GPT-5. *For approximate trends only :-)*

This has become a very proprietary topic!

We'll characterize the high-level decisions in training a model LLM.

Each step offers many opportunities to diverge, but the need for scale tends to make everyone very conservative! *("DL practice is the prior".)*

1. **Architecture:** Modeling language via Decoder-only Transformers.
2. **Pre-training:** Giving LLMs broad knowledge of language, the world, and excellent foundations for reasoning and fine-tuning.
3. **Post-training:** Teaching LLMs to be instruction-following assistants and to be effective at math, coding, using tools, etc.

*Details and **engineering at scale** <u>really</u> matter.*

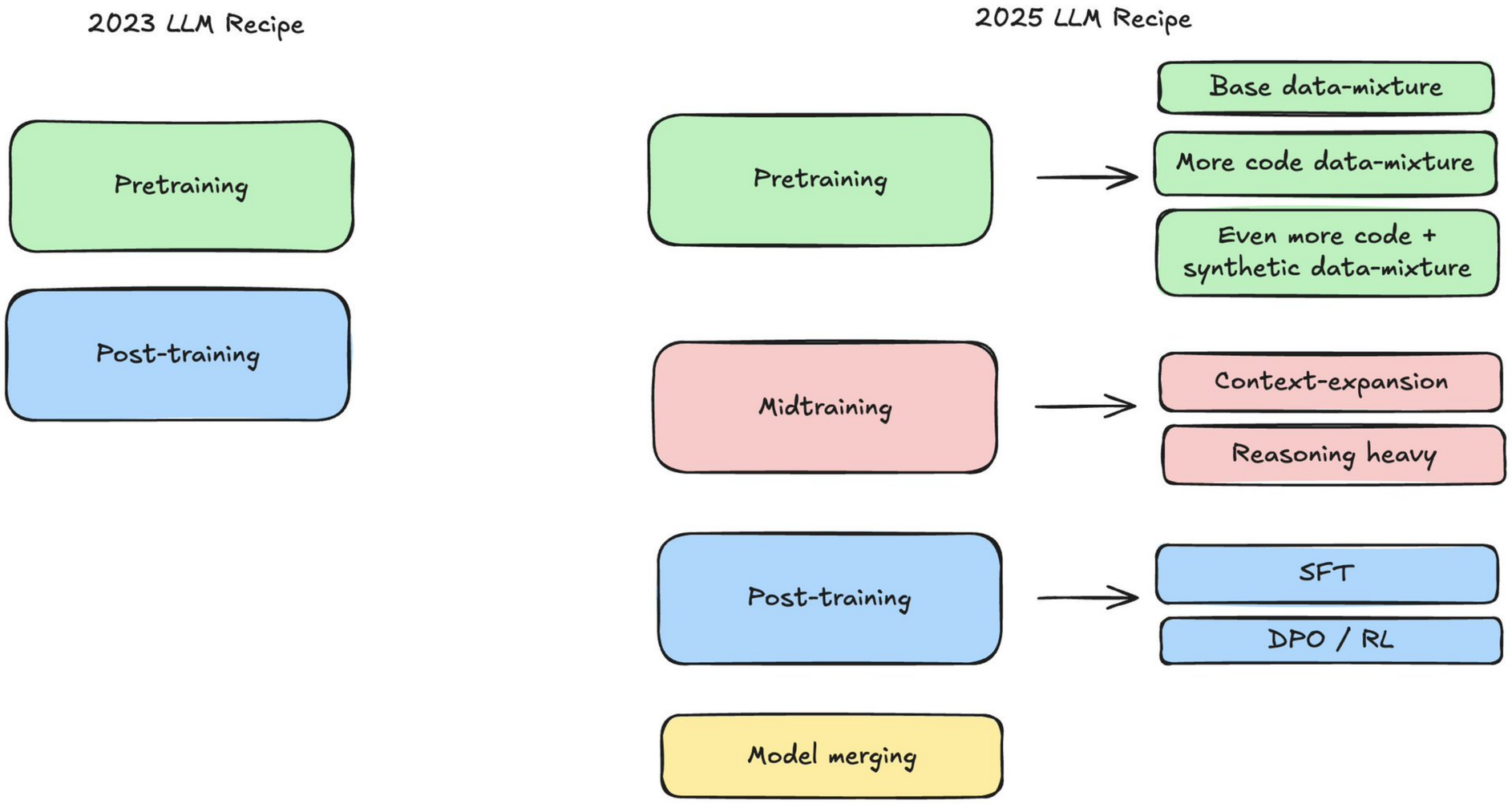


Image Credit: Awni Hannun

Architecture

What's out there we can learn from?

- Open research: Word2Vec (2013), …, Transformer (2017)
- Open models: BERT (2018), GPT-1 (2018), GPT-2 (2019)
- Much more closed research and models: GPT-3 (2020)
- Basically all closed: GPT-4 onwards, Claude, Gemini, …
- Modern open models, varying from **open weights** to **open source**: Llama-{1–4}, Qwen-{1–3}, SmolLM-{1–3}, OLMo-{1,2}, GPT-OSS, …
- Open development: Marin, with *complete logs in public*!

Tokenization for language

- Idea #1: Just use whitespace to determine “words”.
  - “The Eiffel Tower is…!” → [“The”,”Eiffel”,"Tower","is", …] — Why not?
  - Size? Out of vocabulary, like for a rare word or misspelling? Composition (morphology)?
- Idea #2: Characters. “can’t” → [“c","a","n","’","t"]
  - Why not? Long sequences. Inductive bias too weak? (Hmm.)
- Idea #3: Subword tokens. “can’t dislike this” → [“can”,”’t”, “ dis”, “like”, “ this”]
  - Start with the tokens from idea #2. Count adjacent pairs across corpus! Merge the most frequent pair into one new token. Repeat.

Review: Transformer Encoders

BERT, trained via masked language modeling ("fill in the blanks")

2 Challenges?

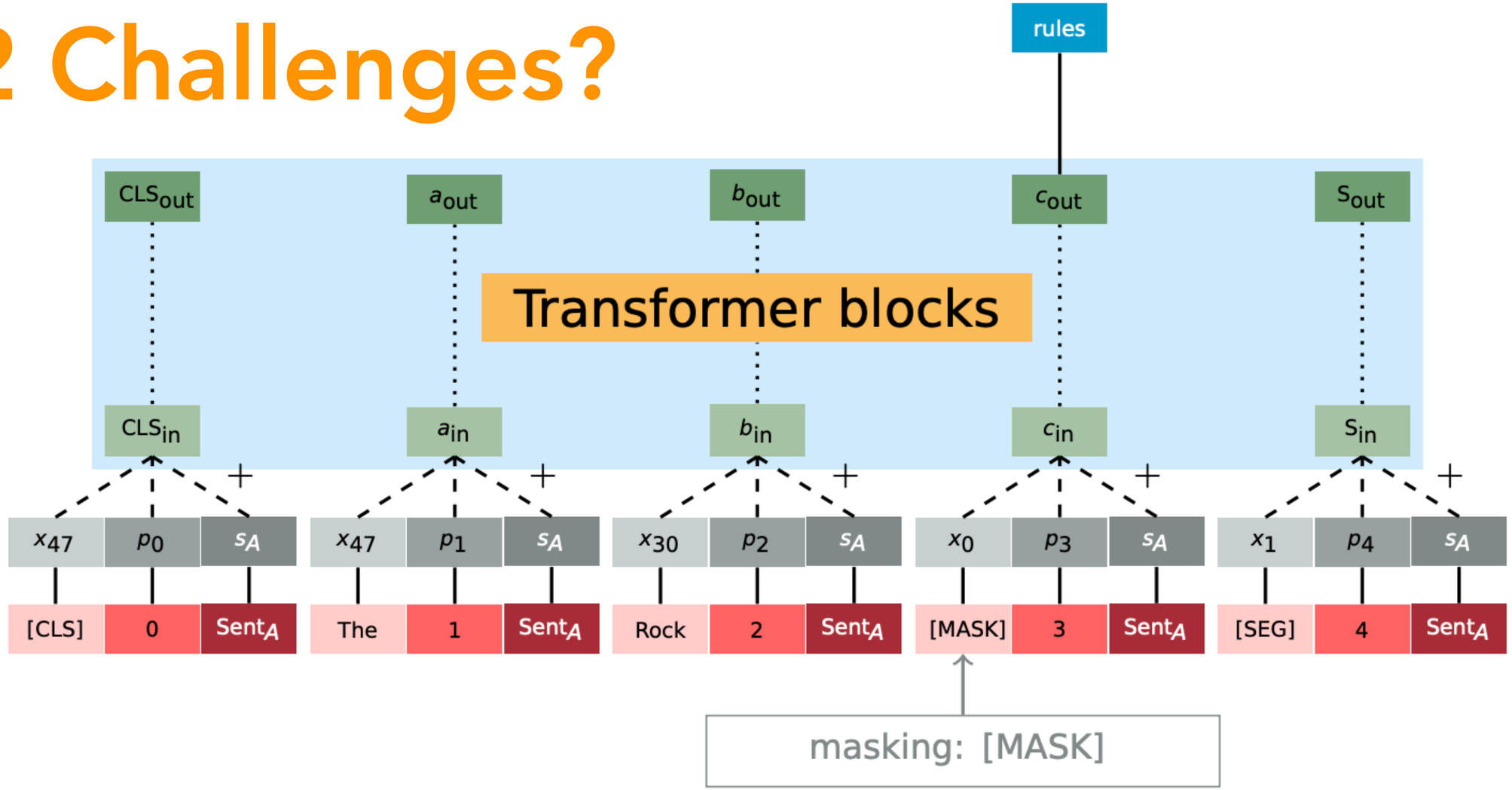
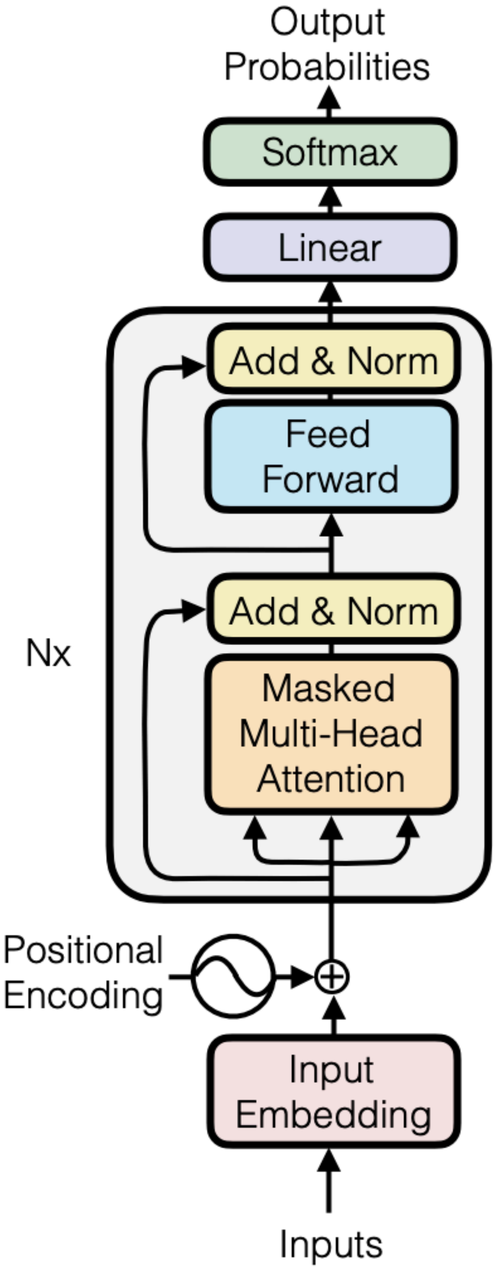


figure: Stanford CS224U (Chris Potts)

Decoder-Only Transformers ("GPT")

- Like an Encoder, but with a **causal mask**: During attention, token *t* can only attend to tokens ≤ *t*.
- At each position, produces a **probability distribution over the next token**. Trained as a *per-step* classifier to predict token x_{t+1} **for all t**.
- In each attention layer, K/V representations of every token *t* are not affected by "future" tokens. This enables fast **autoregressive decoding**.

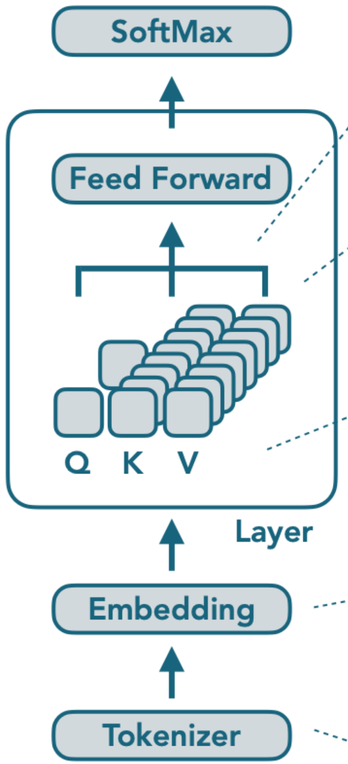


Autoregressive Decoding

1. Tokenize and embed the input prompt.
2. “Prefill”: Run a forward pass over all prompt tokens. Compute attention keys/values for each token and cache them (“KV cache”).
3. Generation Loop, until termination (e.g., EOS token):
    1. **Sample the next token** from the output probabilities at position t.
    2. **Append** new token x_{t+1} to sequence.
    3. Compute **attention K/V/Q for new token** (and update KV cache), while applying attention over cached KV from previous positions.

Example of Architecture Details. For SmolLM3.

Model Anatomy



Grouped Query Attention

The 16 attention heads share 4 query which maintains the performance of full multi-head attention and saves memory during inference.

Intra-Document Masking

Attention masking ensures that tokens from different documents in the same training sequence don't attend to each other which helps with long context training.

NoPE

We use NoPE which selectively removing rotary position embeddings from every 4th layer improving long context performance.

No Weight Decay in Embeddings

We remove weight decay from embedding layers to improve training stability as embedding norms naturally stabilize at healthier values.

Multilingual Tokenizer

We use the Llama 3.2 tokenizer which covers all languages used for pretraining.

Training Configuration

Parameter count: 3.08B
Initialization: N(0, std=0.02)
Layers: 36
Rope theta: 50k

Sequence length: 4096
Batch size: 2.36M Tokens
Optimizer: AdamW (eps=1e-8, beta1=0.8, beta2=0.95)
Learning rate (peak): 2e-4
Gradient Clipping: 1.0
Weight decay: 0.1
Gradient accumulation: 1
Micro batch size: 3
Precision: bf16
Tensor parallel: 2
Data parallel: 192

Throughput: 14k tokens/sec/gpu
MFU: 29.43 %
Training duration: 24 days

Example of Architecture Details. For Marin-8B.

Architecture

We settled on the Llama architecture for the usual reasons: it has been shown to work well, easier to plug into existing inference stacks, no one ever got fired for buying IBM, etc.

We used the same settings as Llama 3.1 8B. More specifically:

Parameter	Value
`hidden_dim`	4096
`intermediate_dim`	14336
`num_heads`	32
`num_kv_heads`	8
`num_layers`	32
`activation_function`	`silu`

We used Levanter's implementation of the Llama architecture. We trained with a sequence length of 4096 tokens per sample. We used JAX's TPU Splash Attention kernel.

We used mixed float32/bfloat16 precision, with parameters and optimizer states in float32 and compute in bfloat16 (except for the final softmax over the vocabulary.) We used the AdamW optimizer. We do not weight decay input embeddings or layer norm parameters.

Pretraining: Data, Hardware, Runs

What can language modeling on Web text do?

- The Eiffel Tower is located in _____, France. [factual knowledge]
- She crossed the street, checking for traffic over ___ shoulder. [syntax & coreference]
- The value I got from the two hours watching this movie was the sum total of the popcorn and the drink. The movie was ___. [sentiment]
- A shirt is $40 but we have a 25% discount; the sale price is $___. [math word problems]
- You are Albert Einstein. You solved this tricky physics problem… [prompting!!]
- def solve(problem): _____ [code generation]

An early bet: The only way an LLM can reliably predict the next token at scale is if it can model the complexity of the phenomenon the text revolves around!

Slightly modified examples from John Hewitt and Chris Manning.

Data is the most important element

- We talked a lot about inductive biases. But we **aren't very good at designing *detailed* inductive biases** for our world and languages.
- **Design them by data!** We're lucky there's so much varied text — at least in English. What would AI in 2026 look like if we didn't…?
- This has been extremely iterative (and heuristic!)…
  - People used to filter out any snippets on the Web that looked like they contained code (e.g., curly braces "{").
  - Now, you pay special attention to having **lots** of good code!

“Train on the Web”

- There’s no central ‘zip file’ of the Web anywhere.
- Must **crawl pages**, starting from seed links!

- The average page on the Web is essentially useless gibberish.
- Must design scalable systems to decide **what to actually keep**.
- Must iterate with **scaling laws** (later).

Most pages on the Web are low quality…

- **What does low quality mean for training your LLM?**

- Idea #1: Over-sample sites we think will be informative!
  - Wikipedia, arXiv, GitHub, StackExchange, …? Reddit?

- Idea #2: Filter based on known heuristics.
  - Upvotes? Links *from* upvoted Reddit posts? PageRank?

- Idea #3: Ask an LLM to “assess quality” and *scale* this.
  - Distill into a tiny model on simple/cheap features.

The pages that remain still need lots of processing.

Can we just dump them right in? Why not?

- Webpages are written for humans…
  - HTML, JavaScript, Boilerplate, …
- There's TONS of duplicated text!
  - How do you define this?
  - Filter in less than N^2 time?

	#Tokens	#HQ tokens	#HQ +%
Trafilatura-filtered	994	80	-
Justext-filtered	1,380	104	28.6%
Justext	1,804	127	57.4%

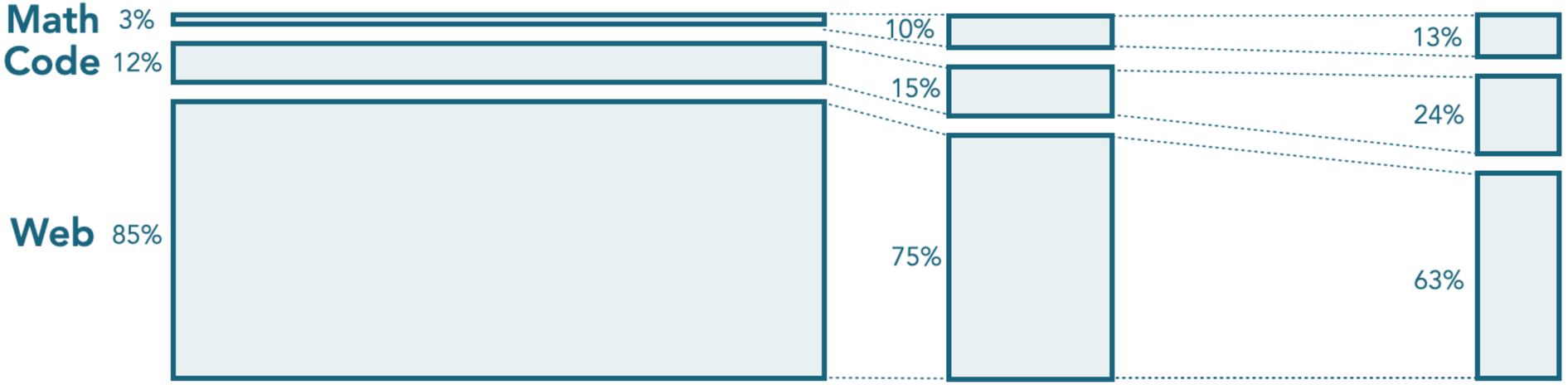
Table 1: Extraction and filteration token count statistics (billion). Tokens counted after deduplication.

HTML-to-text Extraction We test two HTML-to-text extractors, Justext (Pomikálek, 2011) and Trafilatura (Barbaresi, 2021). Qualitatively, we view both extractors at the same level of quality. Quantitatively, we calculate token yields of both extractors on 13 selected snapshots of Common Crawl (see Appendix F). The statistics are reported in Table 1. We see that Justext can yield more tokens, notably more high-quality tokens (+28.6%) by the standard of Fineweb-Edu classifier (score 3, 4, and 5). We highlight that boosting unique token amount is of great importance when building long-horizon pretraining dataset, e.g., 15T tokens for

Nemotron-CC (2024)

Example of Data Details. For SmolLM3.

Pretraining Recipe



Phase I

Description: Base training

Duration: 8T tokens

Datasets: Base mix for pretraining

Web: FineWeb-Edu, DCLM, FineWeb2, FineWeb2-HQ

Code: The Stack v2 (16 langs), StarCoder2 PRs, Jupyter/Kaggle NBs, GH issues, StackExchange

Math: FineMath3+ | InfiWebMath3+

Phase II

Description: High quality injection

Duration: 2T tokens

Datasets: Adding Stack-Edu, FineMath4+, InfiWebMath4+, MegaMath (incl. Qwen Q&A, Pro synthetic rewrites, and text code interleaved blocks)

Phase III

Description: LR Decay

Duration: 1.1T tokens

Datasets: Upsampling high quality code/math datasets and adding instruction/reasoning data such as OpenMathReasoning

LR Schedule

Example of Pre-training Data Details. For OLMo 2.

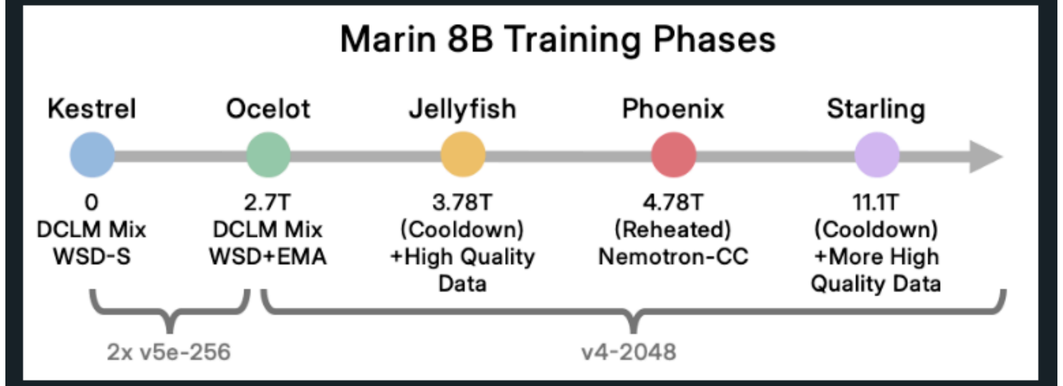
Source	Type	Tokens	Words	Bytes	Docs
Pretraining ✦ OLMo 2 1124 Mix					
DCLM-Baseline	Web pages	3.71T	3.32T	21.32T	2.95B
StarCoder filtered version from OLMoE Mix	Code	83.0B	70.0B	459B	78.7M
peS2o from Dolma 1.7	Academic papers	58.6B	51.1B	413B	38.8M
arXiv	STEM papers	20.8B	19.3B	77.2B	3.95M
OpenWebMath	Math web pages	12.2B	11.1B	47.2B	2.89M
Algebraic Stack	Math proofs code	11.8B	10.8B	44.0B	2.83M
Wikipedia & Wikibooks from Dolma 1.7	Encyclopedic	3.7B	3.16B	16.2B	6.17M
Total		**3.90T**	**3.48T**	**22.38T**	**3.08B**

Example of Run Details. For Marin-8B.

The training process often requires babysitting and some active decision making!

Part of this is due to the extreme costs of a single run in GPU-years (e.g, 48x8-H100s for 24 days for just SmolLM3 at 3B parameters).

Training Phases



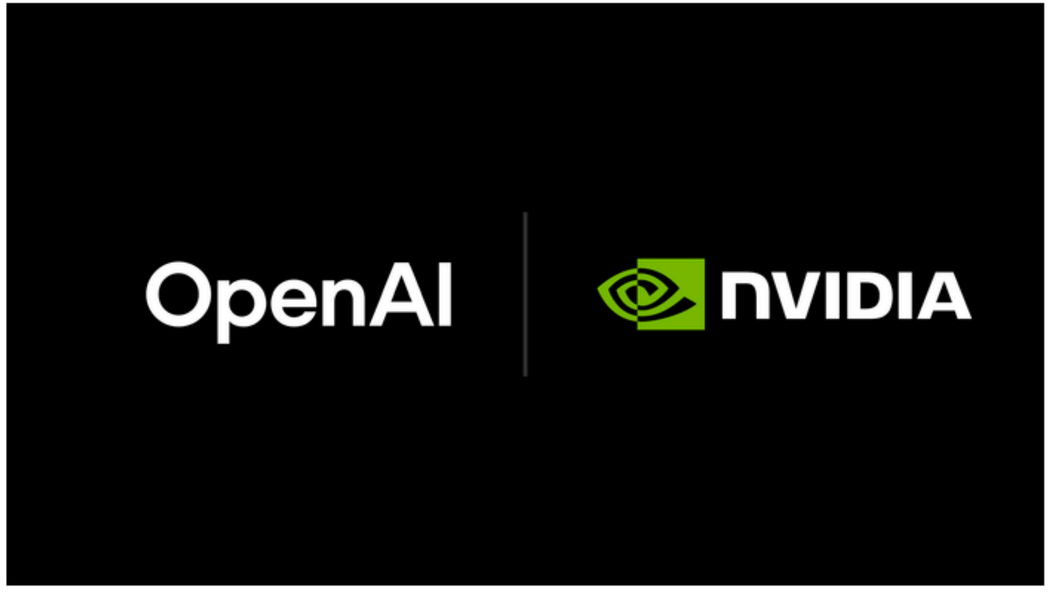
Retrospectively, we can partition the 8B run into several distinct phases—each nicknamed after an animal:

- *Kestrel (DCLM WSD-S Phase)*: In the first phase, we used the "DCLM mix" and WSD-S for about 2.7T tokens. We used 2x TPU v5e-256 coordinated with multislice for this. (0→2.7T tokens)
- *Ocelot (DCLM WSD Phase)*: We were given access to a v4-2048 slice and moved to that. To better utilize the hardware, we increased our batch size 50%. We also switched from WSD-S to WSD. We kept the learning rate high through 3.78T tokens.
- *Jellyfish (First Cooldown)*: It was time to cooldown as we were starting to run low on DCLM. Following recent work on midtraining (e.g. Olmo 2), we decided to fold in higher quality data during cooldown. (3.78T→4.78T tokens)
- *Phoenix (Reheated)*: We had more time for training, so we rapidly rewarmed the model and transitioned our mixture to Nemotron-CC (plus StarCoder Data). (4.78T→11.1T tokens)
- *Starling (Second Cooldown)*: Now we were running low on time, so we started another cooldown. We followed a similar process to the first cooldown, but added a few new datasets that we had created and also some that had dropped since our previous attempt. (11.1T→12.75T tokens)

These phases were not planned in advance. Decisions were made reactively based on changing timelines and data availability.

OpenAI and NVIDIA Announce Strategic Partnership to Deploy 10 Gigawatts of NVIDIA Systems

September 22, 2025

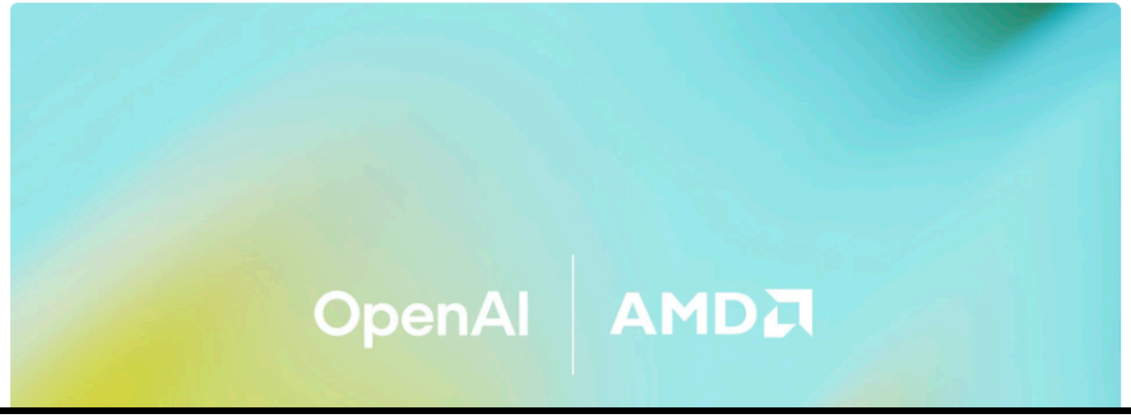


News

- Strategic partnership enables OpenAI to build and deploy at least 10 gigawatts of AI data centers with NVIDIA systems representing millions of GPUs for OpenAI's next-generation AI infrastructure.
- To support the partnership, NVIDIA intends to invest up to $100 billion in OpenAI progressively as each gigawatt is deployed.
- The first gigawatt of NVIDIA systems will be deployed in the second half of 2026 on the NVIDIA Vera Rubin platform.

October 6, 2025 Company

AMD and OpenAI announce strategic partnership to deploy 6 gigawatts of AMD GPUs



Then you may babysit the training runs…

- You monitor the loss over time.
- You watch out for spikes in loss or gradient norm. Stability is hard!
- You also track some evals (hmm).
- Recall: why does deep learning generalize so well?
  In part, because we work so hard and evolve decisions until it does!

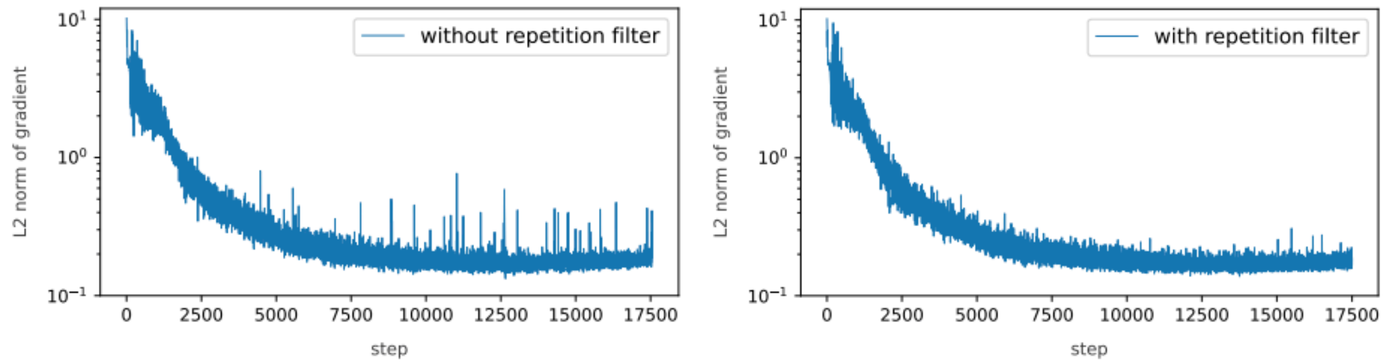


Figure 3 Comparison of the gradient norm for two runs, one without n-gram filter, and one with. Ignoring long repetitive sequences of n-grams eliminates many spikes.

2 OLMo 2 Furious (2025)

Scaling Laws I (motivation)

So many decisions to make…

- I have a fixed GPU budget. Train on more data? Or make model larger?
- **Data:** Which of several different data mixtures do I use?
- **Architecture:** Do I train a Transformer, a *modified* Transformer, an LSTM?
- Other hyperparams too, like optimizer, depth/width, learning rate, …

- Idea #1: Largest model. Train 50 models and pick best on dev! *Challenges?*
  - Then you can only train with 1/50 of your budget…
  - Are 50 runs even enough to cover your different choices?

Idea #2: Tune hyperparams at small scale!

- Then just copy them over and run them at large scale.
- *Challenge?*

- … turns out they don’t transfer so easily! Best choice at small scale may not be the same at larger scale. (E.g., inductive biases or LR)

- This also doesn’t tell you how large to make your choices.

Idea #3: Study "Scaling Laws"

- Train and tune many smaller models (starting from what choices?)
- For each alternative, plot your LM cross-entropy loss as you scale some axis (e.g., data or parameters) on a log-log plot.
- *Extrapolate??*

To be continued…

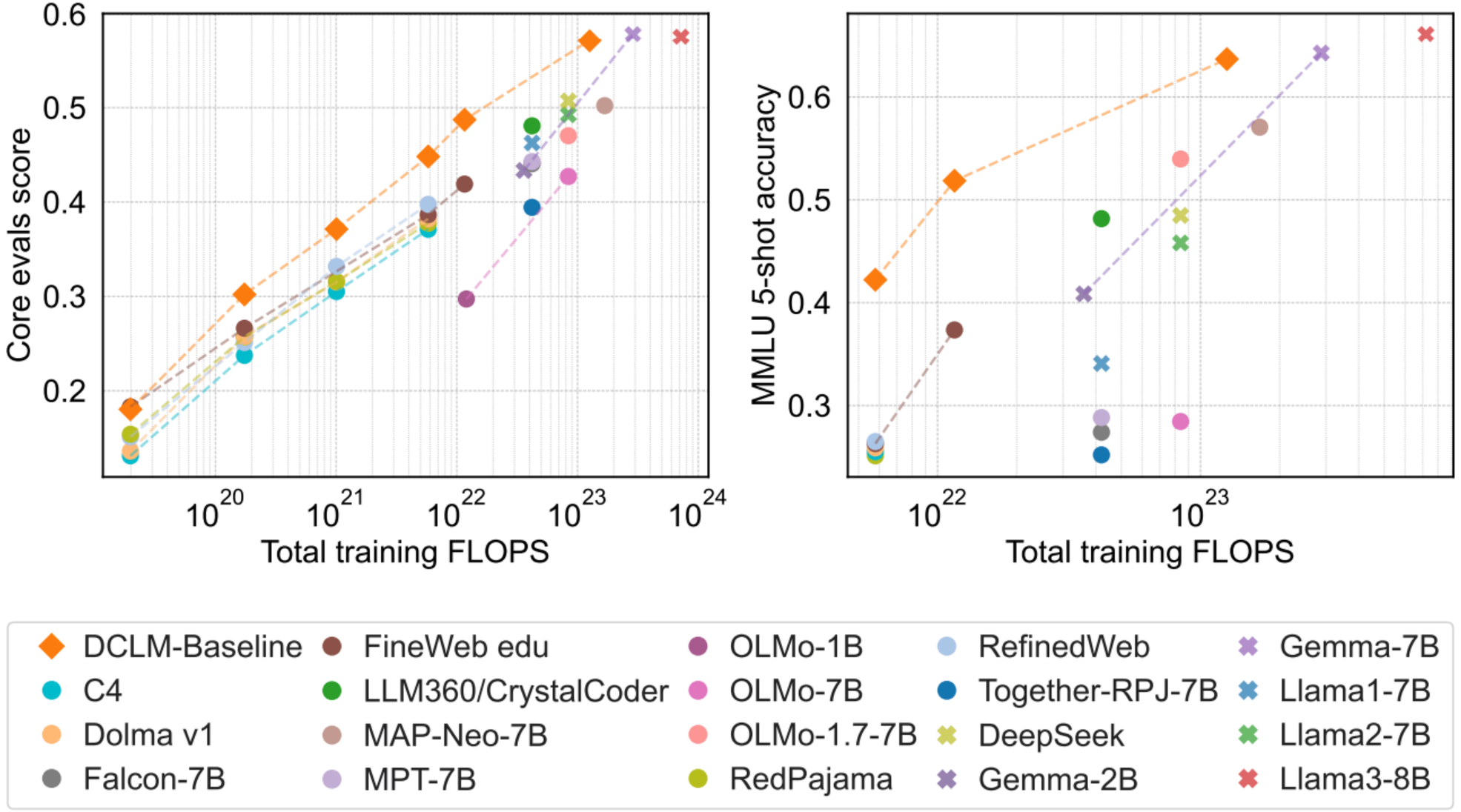
Learning Curves: Asymptotic Values and Rate of Convergence

Corinna Cortes, L. D. Jackel, Sara A. Solla, Vladimir Vapnik, and John S. Denker
AT&T Bell Laboratories
Holmdel, NJ 07733

Training classifiers on large databases is computationally demanding. It is desirable to develop efficient procedures for a reliable prediction of a classifier's suitability for implementing a given task, so that resources can be assigned to the most promising candidates or freed for exploring new classifier candidates. We propose such

Learning Curves: Asymptotic Values and Rate of Convergence (1993!)
Scaling Laws for Neural Language Models (2020)

Related: How do we know **what data to use**?



DataComp-LM: In search of the next generation of training sets for language models (2024)

Some personal takeaways… (though wait for lecture 12!)

- Good ideas are often **simple**! (Occam's razor?)
- But they may only shine when aggressively scaled.
- Doing so well takes **a lot of engineering** and attention to **complex** detail. (No free lunch; gotta adapt to this world's biases. Overparameterization: Simple + Spiky?)
- **But there are often clever methods that scale better!**
- Unless someone consistently champions actively scaling something, like deep learning or LLMs, it won't be done. And you'll never know.